



ABDM FHIR Document Bundle Generator

Open-Source LLM-Driven Clinical PDF → ABDM/NHCX FHIR Bundle Pipeline



What We Built

We designed and implemented a **fully open-source, privacy-preserving, LLM-orchestrated pipeline** that converts clinical PDFs (Diagnostic Reports and Discharge Summaries) into **HL7 FHIR R4 Document Bundles compliant with ABDM / NHCX profiles**.

This is not a simple extraction tool.

This is a **structured, dependency-aware, rulebook-driven FHIR generation framework** powered entirely by local Large Language Models — without using any paid APIs.



Privacy-First Architecture & Data Sovereignty

To ensure the highest standards of data sovereignty and privacy in healthcare, our solution **operates entirely through on-premises microservices**. The architecture runs fully offline and locally, eliminating the need for cloud LLM endpoints or external APIs. This local deployment ensures **zero data leakage** and full health data sovereignty, directly **aligning with the Digital Personal Data Protection (DPDP) Act requirements** for secure and compliant processing*. To manage the complexity of multi-page documents, we use a pre-processing stage called "Patient Grouping". This process uses Docling to extract text and then intelligently groups patients based on identifying attributes such as age, gender, and laboratory collection details. By organizing document content into patient-specific segments before extraction, the system prevents cross-patient data mixing, improves structural clarity, and ensures accurate resource generation within a secure, production-ready environment. By maintaining complete local execution, the system guarantees secure handling of clinical documents while preserving strict ABDM/NHCX structural compliance and UUID linking integrity within a production-ready healthcare environment.



Why Our Approach Is Different

Most PDF-to-JSON systems:

- Extract raw text
- Use one-shot prompts
- Generate loosely structured JSON
- Break when structure becomes complex

Our system instead:

- ✓ Uses **LLM-Orchestrated Dynamic Workflows**
- ✓ Enforces **FHIR Resource Dependency Graphs**
- ✓ Uses **ABDM Rulebooks as Structural Constraints**

- ✓ Groups multi-patient PDFs intelligently
- ✓ Assembles true FHIR *Document Bundles*
- ✓ Embeds original PDF in **DocumentReference**
- ✓ Maintains UUID linking integrity
- ✓ Runs completely **offline and locally**
- ✓ Is 100% **open source**

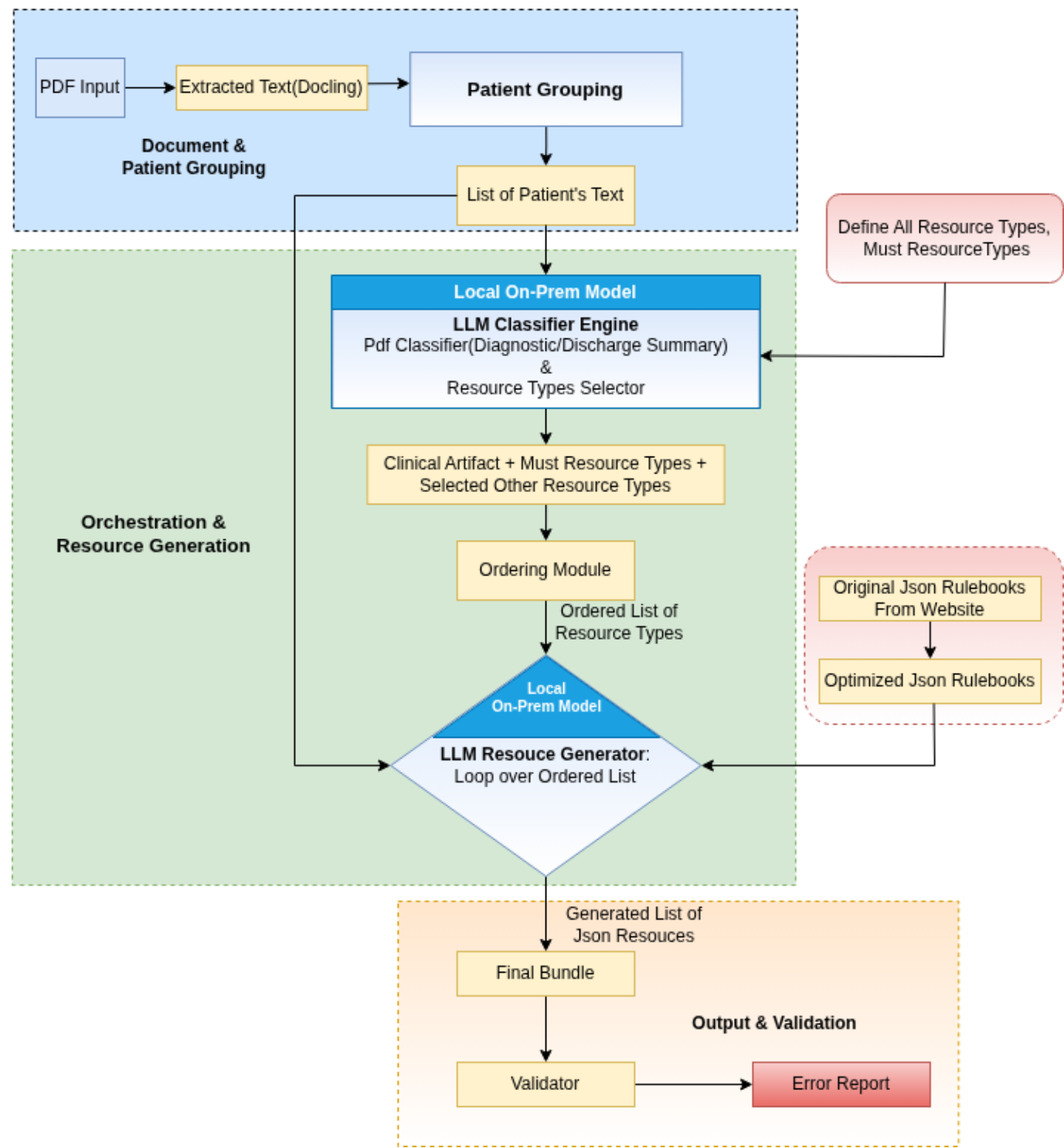
This makes our pipeline significantly more robust and architecturally superior to naive prompt-based extraction.



System Architecture

Problem 2

ABDM FHIR Document Bundle Generator



Core Methodology

1 Structured Pipeline (Not Single LLM Call)

Instead of relying on one large prompt, we designed:

PDF

- OCR (Docling)
- Page Grouping (multi-patient detection)
- LLM Document Classification
- Dynamic Workflow Builder (LangGraph)
- Resource-wise LLM Extraction
- Dependency-aware Linking
- Bundle Assembly
- Post Processing
- Validator

Each FHIR resource is generated independently with:

- Dedicated rulebook
- Strict structural prompt
- UUID enforcement
- Terminology constraints

This significantly improves structural reliability.

2 Rulebook-Driven Extraction

Each resource uses its ABDM structure definition JSON.

This prevents:

- Hallucinated fields
- Wrong nesting
- Invalid structure

However, rulebooks are extremely large.

Smaller LLMs struggle with them in a single pass.

So we:

- Break extraction into resource-level calls
- Use dependency ordering
- Enforce strict output constraints

This modular architecture increases accuracy.

3 LLM Model Strategy

We used:

Qwen 2.5 – 32B

Running locally on:

- NVIDIA A6000
- NVIDIA A6000 ADA

Why?

Because:

- We had limited GPU compute
- Hackathon constraints
- Fully local execution required

Even though Qwen 2.5 32B is a mid-tier model, our architecture compensates for model limitations via structured orchestration.

If we had access to:

- Larger models (70B+)
- More powerful GPUs (H200 class)

Accuracy would significantly improve.

For this problem:

Accuracy > Latency

And our design prioritizes that.

Open Source Advantage

This is one of our strongest achievements.

- No OpenAI API
- No paid APIs
- No cloud inference
- No external data transmission
- Fully local
- Fully open-source stack

This means:

- ✓ Zero data leakage risk
- ✓ Full data sovereignty
- ✓ Deployable inside hospital networks
- ✓ Compliant with healthcare privacy expectations

This makes our framework production-safe for real healthcare systems.

Performance Observations

For a 3-page PDF:

| GPU | Average Time | ||*****|| NVIDIA A6000 | 6 – 8 minutes || NVIDIA A6000 ADA | 5 – 7 minutes |

Stronger GPUs (H100 / H200 class) would reduce latency significantly.

We intentionally did not optimize for speed — we optimized for structure and correctness.

Microservice Deployment

We built a live working platform:

 <https://nhcxhackathon.tanuh.ai/>

Features:

- Upload PDF
- Generate ABDM/NHCX FHIR Bundle
- Download JSON
- Validate bundle
- View validation errors

Supports:

- Problem Statement 2 (Diagnostic / Discharge)
- Problem Statement 3 (Insurance Plan Bundle)

Fully open source backend.

Current Validation Capability

We integrated:

- ✓ HL7 FHIR Validator CLI
- ✓ Error extraction
- ✓ Error reporting in UI

However, due to hackathon time constraints:

We were unable to fully implement automated error correction layers.

Our Big Future Vision: LLM-In-The-Loop Validation

This is our strongest future enhancement idea.

We propose adding **multi-layer validation agents**:

Layer 1 – Missing Field Resolver

LLM compares:

- Extracted text
- Generated JSON

If fields exist in text but are missing in JSON (e.g., Observations), it regenerates them.

Layer 2 – Terminology & Coding Fixer

LLM reviews:

- SNOMED
- LOINC
- UCUM
- Profile conformance

Corrects:

- Wrong code systems
 - Invalid coding formats
 - Terminology inconsistencies
-

Layer 3 – Structural FHIR Validator Loop

We design an iterative loop:

1. Generate JSON
2. Run FHIR Validator
3. Feed errors back to LLM
4. LLM fixes JSON
5. Repeat K times ($k = 3$ or 4)

This "LLM in the loop" architecture:

- Backpropagates validation errors
- Improves structural conformance
- Produces increasingly accurate bundles

We could not complete this due to time constraints.

But architecturally — this is extremely powerful.

Why Our Approach Is Strong

Even large LLMs struggle with:

- Large rulebooks
- Deeply nested FHIR structures
- Multi-resource linking
- UUID reference integrity

Our pipeline solves this via:

- Resource decomposition
- Dependency graph ordering
- Controlled prompt templates
- Assembly-first architecture
- Composition-first bundling

This is not just extraction.

This is structured healthcare data engineering.



Known Limitations

- High latency due to multi-step extraction
- Validation auto-correction not yet implemented
- Heavy GPU requirement for 32B model
- Large rulebooks stress model context window



Supported Clinical Artifacts

- DiagnosticReportRecord
- DischargeSummaryRecord



Execution Flow

main()

- get_abdm_json()
 - OCR & Page Grouping
 - LLM Classification
 - Dynamic Workflow (LangGraph)
 - Resource Nodes
 - Assembly Node
 - Clean & Reorder
 - DocumentReference Embedding
 - JSON Output
-



System Architecture

PDF Input

- Docling OCR
- Patient Grouping
- LLM Classification
- Dependency Graph Builder
- Resource Agents
- Bundle Composer
- Validator
- Final ABDM JSON



Why We Believe This Is Production-Capable

- Deterministic workflow
- Strict structural prompts
- UUID enforcement
- Composition-first bundling
- Embedded source document
- Fully local execution
- Microservice deployment ready

This is a strong foundation for real-world healthcare integration.



Setup Instructions

Clone Repository

```
git clone <repo-url>  
cd <repo-folder>
```

Create Virtual Environment

```
python -m venv venv  
source venv/bin/activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Install Ollama

<https://ollama.com>

```
ollama pull qwen2.5:32b
ollama serve
```

Run Pipeline (CLI)

```
python main.py input.pdf
```

Run as Microservice

```
uvicorn main:app --reload
```



System Requirements

- Python 3.10+
- 30–32GB RAM recommended
- GPU strongly recommended
- NVIDIA A6000 class minimum for 32B model



Conclusion

We built:

- A structured LLM-orchestrated ABDM FHIR generator
- A dynamic dependency-aware pipeline
- A privacy-preserving open-source framework
- A deployable microservice
- A validation-ready architecture
- A future-ready LLM-in-the-loop correction system

With more time and stronger GPUs, this system can become:

A fully autonomous clinical-to-FHIR transformation engine with near-perfect structural conformance.